

JAVA INTRO

Friday, July 19, 2024 11:48 PM

ABOUT JVM ARCHITECTURE

CLASS LOADERS

Def : its a subsystem of JVM used to load class files,3 major functions are loading,linking & initialisation.

JVM MEMORY

Method Area : THE JVM area stores class structure,such as the meta data,the class name,the variable,the code for methods,and the constant runtime pool.

Only **one method area per JVM** is shared **among all classes**

Heap :

Most cmn mem type shared across multiple threads,stores all objects & their related instance variables, & arrays.

JVM Language Stacks :

These store local variables along with their results and play an important role in method invocation and return

A new frame gets created on method invocation =,frame gets deleted after the process of method invocation completes

PC Registers :

A PC Register stores the address of the jvm instruction which is currently being executed

Each thread has a separate PC Register

Native method stacks

Execution Engine

Native method interface

Native method libraries

Working :

.java(source code) --> javac(compiled code) --> .class (byte code) , JVM (INTERPRETER FOR WINDOWS)

A class may depend on other class for compilation and execution

The env var class path points to the folders of .class files for the jdk or jvm to refer to

Only a corresponding jvm can understand the bytecode

Jvm interprets the bytecode to real machine instructions

Real machine executes instructions to run the program

BASIC JAVA PRGM STRUCTURE

Public class Myfirst

```
{
    Public static void main(String [] args)
    {
    }
}
```

Every java program has at least one class with main() method
The main() method's signature is predefined
Name of classes and methods follow camel casing, class name starts with a capital letter and method name starts with a small letter
Source file is generally named after the class name
Print and println take string as a parameter

OBJECT

Groups related states and behaviours and is often used to model real world objects.
Object is a key to understand object oriented tech,
State represents the object's data (value)
Behaviour : functionality of an object.
--> Can be referred as a class instance
Exposes behaviour through methods (functions) and stores the state in fields (variables).
Thus an object is an encapsulation of state and behaviour
Represents instance of a class and resides in a specific memory location, **EACH OBJECT NOT ONLY OCCUPIES A DISTINCT PORTION OF MEMORY BUT ALSO HAS A UNIQUE ADDRESS WHICH POINTS TO ITS LOCATION WITHIN THE MEMORY**

OBJECT'S STATE AND BEHAVIOUR CAN BE ACCESSED WITH DOT (.) OPERATOR

Classes

It is a logical entity which includes a group of objects sharing common behaviour and props
It is a blueprint for creating/instantiating individual objects, it's a template that describes the behaviour and state that the object of its type can support

Class does not consume any space, uses class keyword to define a class, **new** keyword is used to create/instantiate objects of the class

DECLARING CLASSES :

Single public class (probably name of the file), caps begin,
If class contained in a package first package statement & next the imports
Package --> import --> class
Class can implement one or many interfaces as per the requirement

Declaring member variables :

Member / instance variables are associated with an object, accessible to all of its methods and member functions
Declared inside the class **BUT Outside of Methods, Constructors, or any block.**

It has access modifiers (public, private, protected, default) they have a name and type (obv)

DEFINING METHODS :

METHOD declaration has following components
Modifier
Return type
Camel casing with first letter in lower case
Parameter list
Methods accessible outside of class are public, avoid private methods as much as possible

PACKAGES

COLLECTION OF CLASSES ,provides access protection and namespace management
Classes declare the package which they belong to

NAMING CONVENTION :

Package name follows dot notation with an inverted domain name

Example : com.deloitte.myproject.

Com.mywebsite.newproject

Package 1st , import next (when u use a class from diff package),

Package makes easier to find classes and interfaces

Avoid naming conflicts (ex : java.lang,java.io,java.util)

All classes from a package can be imported using a wild card (ex : import com.deloitte.myproject.*;)

Public Class Stdnt

```
{
    Private int id;
    Private
    String name;
}
```

P s v m (s ards)

```
{
    Stdnt std = new Stdnt();
    Std.id = 483;
    Std.name = "Hemanth";
}
```

NAMING CONVENTIONS AND STANDARDS :

Package Names	- Should be in lowercase, with a reversed domain name as a prefix - Example: com.example.project
Class Names	- Should be in UpperCamelCase - Example: MyClass
Interface Names	- Should be in UpperCamelCase, and typically should be an adjective - Example: Serializable
Method Names	- Should be in lowerCamelCase - Example: calculateInterest()
Variable Names	- Should be in lowerCamelCase - Example: numberOfStudents

Constants	- Should be in uppercase, with words separated by underscores - Example: MAX_VALUE
Indentation and Formatting	Use 4 spaces for indentation and follow a consistent style throughout the code
Comments	Use meaningful comments sparingly to explain complex algorithms, design decisions, or to provide context where necessary
Braces	Use the "Egyptian brackets" style, with the opening brace on the same line as the declaration
Naming Conventions for Packages, Classes, and Methods	- Follow meaningful and descriptive names that accurately reflect the purpose of the entity - For example, a class that represents a student in a school management system

Naming Conventions (Cont.)

```
package com.example.schoolmanagement;

public class Student {
    private String studentName;
    private int studentId;

    public Student(String studentName, int
studentId) {
        this.studentName = studentName;
        this.studentId = studentId;
    }

    public String getStudentName() {
        return studentName;
    }

    public void setStudentName(String
studentName) {
        this.studentName = studentName;
    }
}
```

```
public int getStudentId() {
    return studentId;
}

public void setStudentId(int studentId) {
    this.studentId = studentId;
}

public void displayStudentInfo() {
    System.out.println("Student Name: " +
studentName);
    System.out.println("Student ID: " +
studentId);
}
```

CODING STANDARDS:

Naming conventions , indentation and formatting , comments , braces .

The basic rule for creating JavaDoc comments is that they **begin with /**** and **end with */**
Use egyptian brck style

Imports

Organize import statements alphabetically and avoid using wildcard imports ("import java.util.*").

Code Duplication

Minimize code duplication by extracting common functionality into methods or classes.

Error Handling

Handle exceptions appropriately, either by catching them or declaring them in the method signature.

Use of Final

Use the 'final' keyword for constants, method parameters, and variables that should not be reassigned.

Use of Enums

Use enums for representing a fixed set of constants.

Code Duplication

Minimize code duplication by extracting common functionality into methods or classes.

Testing

Write unit tests to verify the correctness of your code.

Security

Follow best practices for security, such as avoiding hardcoded sensitive information and sanitizing inputs.

Summary

Here are the key learning points of the module:

- Java programming language is widely used in the distributed Internet environment.
- The key features of Java include garbage collection, robust and secure, garbage collection, static, portable, object-oriented, distributed, multithreaded, and platform-independent.
- JDK is a combination of JRE and other tools, including compilers, debuggers, and several libraries.
- JVM contains stacks that support native methods used in the application. For every thread, a separate native stack is created.
- Java coding standards help maintain consistency and readability across codebases.

JAVA LANGAUAGE PROGRAMMING CONSTRUCTS

Saturday, July 20, 2024 7:28 PM

Java Language Programming Constructs :

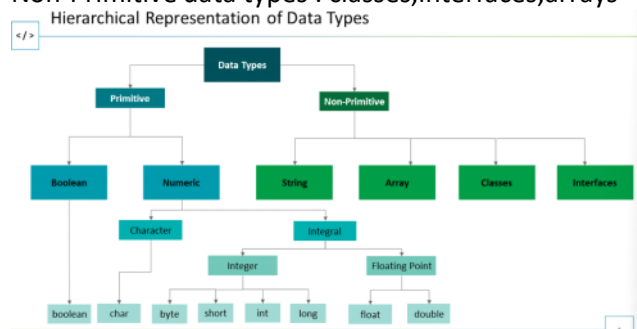
Topic	Descriptions	Duration
Java Language: Programming Constructs	Data Types and Variables and Their Scope, Java Return Types, Operators, Expressions, Blocks, and Statements, Control Flow Statements, and Keywords	12 hours
Control Flow Statements	If, if else, switch, for, while, do while, enhanced for, break, continue	
Modifiers	Access Modifiers, Non-access Modifiers, and Static keyword	

By the end of this session, you will be able to:

- Describe the basic constructs of Java programming language including data types, variables, operators, functions, and control flow statements
- Analyze the given problem-statement/use case, identify, and make use of appropriate Java language constructs for programming robust solutions in Eclipse
- Outline the use of access and non-access modifiers for defining/managing access mechanisms in Java

Prim : Boolean,char,int,short,long,float,double

Non-Primitive data types : classes,interfaces,arrays



Variables—Types and Scope

Variable Types	Descriptions
Local Variable	Declared inside the body of the method
Instance Variable (non-static fields)	Declared in the class but outside the method body
Class Variable (static fields)	Declared as static

Example:

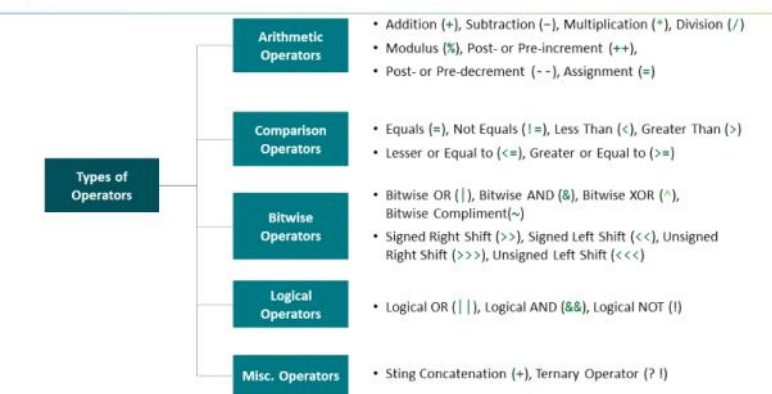
```
class A {
    int data = 50; // instance variable
    static int m = 100; // static variable
    void method() {
        int n = 90; // local variable
    }
} // end of class
```

Java Return Types

'return' is a keyword in Java that serves as an instruction for a program to exit a specific method. The exit may occur with the method either providing a value back to the part of the program that called it or without returning any value.

- It is mandatory for every method in Java to be declared with a return type.
- A method that does not return anything is declared as a return type (void).
- Methods are declared with the return type that corresponds to the type of value returned by a method.
- Methods may have return types such as:
 - primitive value (int, float, double)
 - class object
 - current or same-class object
 - variable

Operators



Expressions and Blocks

Expressions	Blocks
• A construct is made up of variables, operators, and method invocations. • The entire construct that evaluates to a single value is called an expression. Example: <pre>int numWheels = 4; System.out.println("No. of wheels in the car are " + numWheels); int area = length * breadth; if (value1 == value2) System.out.println("value1 == value2");</pre>	• Blocks are a group of statements between curly braces. • Blocks can consist of zero or more statements and be placed anywhere a single statement is allowed. Example: <pre>boolean condition = true; if (condition) { // Block 1 - Begins System.out.println("Condition is true."); } // Block 1 - Ends else { // Block 2 - Begins System.out.println("Condition is false."); } // Block 2 - Ends</pre>

Statements

Expression statement, declaration statement, control flow statement

Statements		
Expression Statement	Declaration Statement	Control Flow Statement
<pre>// assignment statement aValue = 8993.254; // increment statement aValue++; // method invocation statement System.out.println("Hello Java!"); // object creation statement Bicycle myBike = new Bicycle();</pre>	<pre>double aValue = 8993.254; int i = 400; long ssn = 456_33_41551;</pre>	<pre>if (expression){ } if (expression){ }else{ } switch(expression){- };</pre>

Statements

Expression Statement
Declaration Statement
Control Flow Statement

```
// assignment statement
aValue = 8993.254;
// increment statement
aValue++;
// method invocation statement
System.out.println("Hello Java!");
// object creation statement
Bicycle myBike = new Bicycle();
```

```
double aValue = 8993.254;
int i = 400;
long ssn = 456_33_4155L;
```

```
if (expression){
}
if (expression){
}else{
}
switch(expression){_
};
```

Copyright © 2024 Deloitte Development LLC. All rights reserved.

Control flow statements

Decision making statements, looping statements, branching statements,

Control Flow Statements

There are three kinds of control flow statements:



Decision-Making Statements

Based on the results of a condition, decision-making statements in Java allow you to make a decision.

- If-then
- If-then-else
- Switch



Looping Statements

A set of instructions repeatedly executed until a certain condition is filled is a loop in programming.

- While
- Do-While
- For



Branching Statements

When a certain condition is met a branching statement allows us to exit from a control statement.

- Break
- Continue
- Return

Switch is req,uk if and if-else

(first small)

```
Char grade = 'c';
```

```
Switch (grade) {
```

```
Case 'A' :
```

```
    Syso();
```

```
    Break;
```

```
Case 'B' :
```

```
    Syso();
```

```
    Brk;
```

```
    .
```

```
    .
```

```
Default :
```

```
    Sys0();
```

```
}
```

For,while,dowhile

The 'while' Statement:

Until the specified condition is true, 'while' is used to iterate a part of the program.

```
int count = 1;
while (count < 11) {
    System.out.println("code in while loop");
    count++;
}
```

The 'do-while' Statement:

Checks for the condition after executing the statements, like the while loop.

```
int count = 1;
do {
    System.out.println("code in do-while loop");
    count++;
} while (count < 11);
```

The 'for' Statement:

In Java, a 'for' statement, also known as a 'for' loop, is a control flow statement that allows a portion of code to be executed repeatedly based on a specified condition. This is particularly useful when we need to perform a task multiple times, such as iterating through an array or a collection.

```
for(int i=1; i<12; i++){
    System.out.println("Count is: " + i);
}
```

```
int[] numbers = {1,2,3,4,5,6,7,8,9,10};
for (int item : numbers) {
    System.out.println("Count is: " +
        item);
}
```

Breaks terminates the for,while,or dowhile loop,continue statement skips the crnt iteration of the for while and dow hile

Return exits from the crnt method and return the control flow to where it was invoked

MODIFIERS :

Access Modifiers and Rules

</>

- There are three access modifiers—public, protected, and private.
- There are four access levels—public, protected, default, and private.
 - **Public:** Public modifier access level can be accessed from within the class and package and outside the class and package. It is accessible everywhere.
 - **Private:** Private modifier access level is only within the class and cannot be accessed outside of the class.
 - **Default:** The default modifier access level is only within the package and cannot be accessed outside of the package. This is the default access level is you do not specify.
 - **Protected:** Protected modifier access level is within and outside the package through a child class. Without the child class it cannot be accessed from outside the package.

Illustration:

Modifier	Class	Package	Subclass
public	Y	Y	Y
private	Y	N	N
default	Y	Y	N
protected	Y	Y	Y

- Classes can have only public or default access.
- The class visibility revolves around whether the code in a class can:
 - Create an instance of another class;
 - Extend (or subclass) another class; and
 - Access methods and variables of another class.
- Some best practices that you must follow:
 - Make the classes public.
 - Make the fields (state) private.
 - Make the methods public.

Non-Access Modifiers and Rules

/>

In order to achieve many functionalities, Java provides several non-access modifiers, including:

- Static modifier: Creates class methods and variables
- Final modifier: Finalizes the implementations of classes, methods, and variables
- Abstract modifier: Creates abstract classes and methods
- Synchronized and Volatile modifiers: Are used for threads

Static variables : belong to class rather than any specific instance.this means all instances the same variable.

Static methods : can be called without creating an instance of a class,they can access static variables and other static methods.

Eg :	<pre> Public class Test{ static int x 100; static void increment(){ Counter++; } } </pre>
------	---

```

Final modifier : final int MAX = 100;
Public final void display() {
    System.out.println("Cannot be changed ");
}

```

Final classes cannot be subclassed

Abstract Modifier : **abstract classes** : cannot be **instantiated** and are **meant to** be **subclassed**, and abstract methods must be implemented by subclasses

Ex:

```

Abstract class Animal {
    Abstract void makeSound();
}

```

```

Class Dog extends Animal{
    Void makeSound(){
        System.out.println("Bark");
    }
}

```

Java Static Variable

- Also known as a class variable, is declared with the static keyword in a class, but outside a method, block, or constructor
- Has only one copy per class, irrespective of the number of objects that are created from it
- Is rarely used apart from being declared as a constant. A constant variable is declared as public/private, static, and final. The initial value of a constant variable never changes.
- Is stored in static memory which is also known as the method area
- Is created when the program starts and is destroyed once the program stops
- Has the same visibility as an instance variable. The visibility modifier (public, private, protected, or none) is chosen based on the specific requirements of the class and its interaction with other classes
- Is accessed using ClassName.VariableName

OOPS

Sunday, July 21, 2024 9:59 PM

Topic	Descriptions	Duration
Java Constructors and this Keyword	Constructors, Constructor Chaining, and this Keyword and its Usage	8 hours
JAVA—OOPS Concepts	Object-Oriented Programming (OOP) Concepts in Java, Constructors, Polymorphism, Inheritance, Abstract Classes, and Encapsulation, Nested Classes, Abstract Classes, Interfaces	
Java Arrays	Java Arrays	

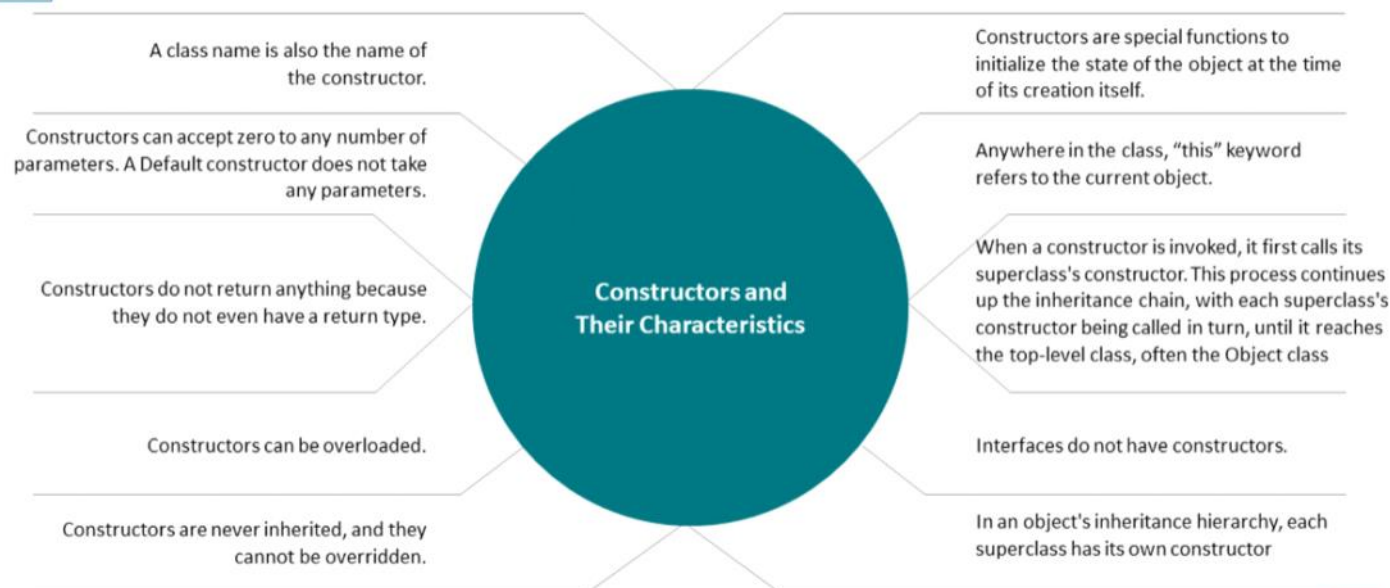
Learning Objectives

By the end of this session, you will be able to:

- Explain the fundamental concepts of constructors, this keyword, object-oriented programming including abstraction, encapsulation, polymorphism, and inheritance
- Describe nested classes, interfaces and abstract classes in Java
- Illustrate the use of constructors in Java along with this keyword
- Demonstrate the use of super keyword in Java inheritance
- Analyze the given problem-statement/use-case, identify, and make use of appropriate Java language constructs for programming robust solutions in Eclipse
- Discuss the concept of arrays for storing multiple values in a single Java variable along with their shortcomings
- Demonstrate the declaration, definition, and use of arrays in Java programs

Constructors

</>



Constructor Types

In Java, constructors can be categorized into two main types:

Default Constructor:

This constructor is provided by Java if no other constructor is explicitly defined in the class. It initializes the object with default values.

Parameterized Constructor:

This constructor allows you to pass parameters when creating an object, enabling you to initialize the object with specific values.

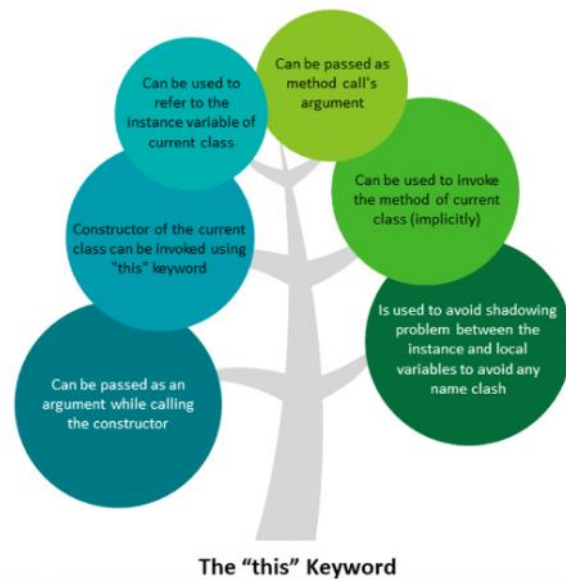
Constructor Chaining

- Refers to the process of one constructor calling another constructor in the same class or in its superclass
- Is typically done using the `this()` or `super()` keyword

Using this()

- In a class, you can have multiple constructors with different parameter lists.
- You can use the `this()` keyword to invoke one constructor from another constructor in the same class.
- This allows you to reuse initialization code.

```
public class MyClass {
    private int x;
    private int y;
    public MyClass() {
        this(0, 0); // Calls the other constructor with default values
    }
    public MyClass(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```



Using super()

- If a subclass constructor does not explicitly call a superclass constructor using `super()`, the superclass's no-argument constructor is called implicitly.
- However, if the superclass does not have a no-argument constructor, the subclass constructor must explicitly call one of the superclass constructors using `super()`.

```
public class SubClass extends SuperClass {
    private int z;

    public SubClass(int x, int y, int z) {
        super(x, y); // Calls the superclass constructor with parameters
        this.z = z;
    }
}
```

Constructor Chaining Example

```
// Superclass
class Vehicle {
    private String type;

    // Constructor of Vehicle
    public Vehicle(String type) {
        this.type = type;
        System.out.println("Vehicle type: " + type);
    }
}

// Subclass
class Car extends Vehicle {
    private String brand;
    private String model;

    // No-argument constructor
    public Car() {
        this("Unknown", "Unknown"); // Chaining to another constructor in the same class
    }
}
```

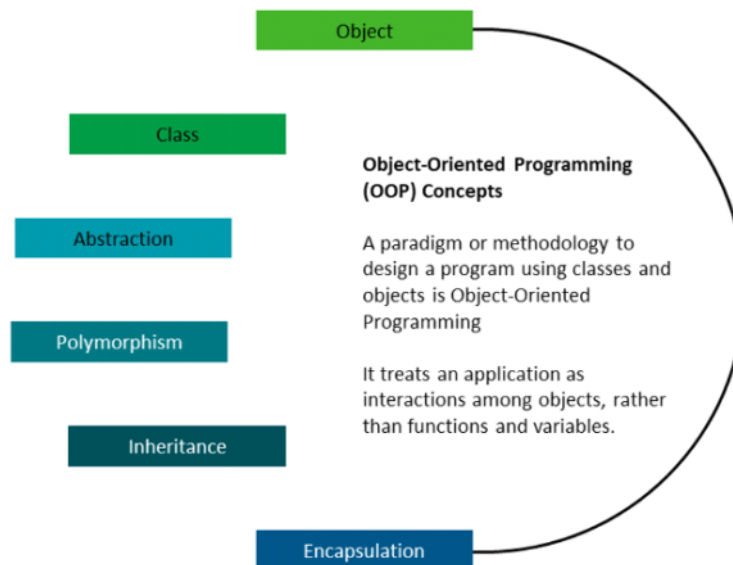
Constructor Chaining Example (Cont.)

```
// Constructor with one argument
public Car(String brand) {
    this(brand, "Unknown"); // Chaining to another constructor in the same class
}

// Constructor with two arguments
public Car(String brand, String model) {
    super("Car"); // Calling superclass constructor
    this.brand = brand;
    this.model = model;
    System.out.println("Car brand: " + brand + ", model: " + model);
}

public static void main(String[] args) {
    // Creating Car objects using different constructors
    Car car1 = new Car();
    Car car2 = new Car("Toyota");
    Car car3 = new Car("Honda", "Civic");
}
}
```

Object-Oriented Programming Concepts



OOP Principles

</>

	Definitions	How Do We Achieve It in Java?
Abstraction	- Hides internal details of an object from the user and shows only the relevant data - Example: Logging into an online bank account	- To achieve abstraction, use the abstract class and interface - Abstract means incomplete (The login verification process is abstracted from you).
Polymorphism	- Has the ability of a method to be used in different ways, which means it can be in different forms at different times (poly + morphos) - The two types of polymorphism are compile-time polymorphism and run-time polymorphism.	- Use method overloading and method overriding - The compile-time (static) polymorphism occurs when a method is overloaded whereas the run-time (dynamic) polymorphism occurs when the actual method itself is changed.
Inheritance	- Refers to when an object acquires (or inherits) the parent's object behaviors and properties - Allows the reuse of codes and improves reusability - Use the "extends" keyword to inherit a class.	- Use extends keywords to declare inheritance. - The parent class is called the base class or super class. - Derived class, Subclass, or Child class- Child class extending the base class.
Encapsulation	- Binds object state (fields) and behavior (methods) together - Protects the integrity of the data - Encapsulation is often confused with data abstraction.	- Use private variables to achieve encapsulation. - Java bean is the fully-encapsulated class because all the data members are private here.

Inheritance—Illustration

/>

Inheritance

```
class SuperClassA {
    public void foo(){
        System.out.println("SuperClassA");
    }
}
class SubClassB extends SuperClassA{
    public void bar(){
        System.out.println("SubClassB");
    }
}
public class Test {
    public static void main(String args[]){
        SubClassB a = new SubClassB();
        a.foo();
        a.bar();
    }
}
```

- Inheritance in Java can be implemented using the *extends* keyword.
- Superclass* is the object being inherited and the *subclass* is the object that inherits the *superclass*.
- A class can extend only one *superclass*.
- Java does not support multiple inheritance; however, a class can implement multiple interfaces.

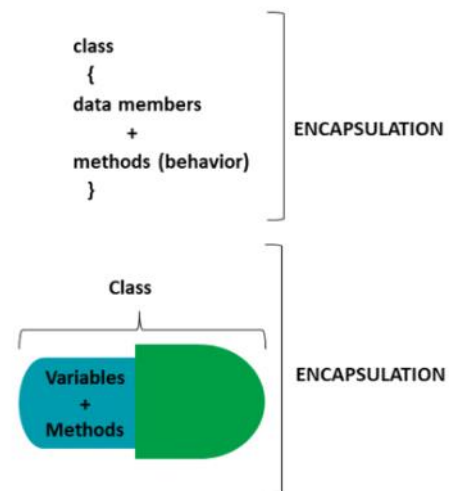
Encapsulation

</>

Encapsulation in Java is a way to wrap the data (variables) and the code acting on the data (methods) together—as a single unit.

Advantages:

- Data Hiding:** Hiding the implementation details in order to restrict the access of the data members.
- Increased Flexibility:** It enables you to make the variables of the class read-only or write-only, depending on the requirement.
- Reusability:** Encapsulation also improves the reusability and is easy to change with new requirements.
- Easy to Test:** Encapsulated code is easy to test for unit testing.



Polymorphism—Method Overloading and Method Overriding—Illustration

/>

Method Overloading

```
public class Circle {
    public void draw(){
        System.out.println("Circle with basic color");
    }

    public void draw(int diameter){
        System.out.println("Circle with basic color
and diameter");
    }

    public void draw(int diameter, String color){
        System.out.println("Circle with provided
parameter color and diameter");
    }
}
```

Method Overriding

```
interface Shape {
    public void draw();
}
class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing circle");
    }
}
class Square implements Shape {
    public void draw() {
        System.out.println("Drawing Square");
    }
}
class Test{
    static Shape getShape() {
        return new Circle();
        // return new Square();
    }
    public static void main(String[] args) {
        Shape sh1 = getShape(); // logic to determine shape
        sh1.draw();
    }
}
```

Abstract Classes

An abstract class can have abstract methods, concrete methods and constructors. It can't be instantiated.

The "abstract" keyword is used to declare an abstract class that does not support multiple inheritance.

A class with abstract methods must be declared abstract.

Abstract classes have partial implementation, but they are meant for further extension by subclasses.

A concrete method has a body compared to an abstract method which does not have a body.

An abstract class can have final, instance, static variables.

Abstract
Class

Abstract Class—Illustration

/>

Abstract Class

```
abstract class Bike{
    abstract void run();
}
```

```
class Bullet extends Bike{
    void run(){System.out.println("running");}
    public static void main(String args[]){
        Bike obj = new Bullet();
        obj.run();
    }
}
```

/>

Abstract vs. Final Classes

Classes can be final and abstract, but they cannot be declared both final and abstract at a time.

Do you know that a final class cannot be subclassed, and an abstract class cannot be instantiated?

A single abstract method in a class implies that the whole class must be abstract.

The first concrete class to extend an abstract class must implement all the abstract methods.

Interfaces

A Java interface is used to achieve abstraction. However, they cannot be instantiated.

By default, all the methods() inside an interface are public abstract methods.

By default, all the variables declared inside an interface are public static final and must be initialized.

Interface cannot have a constructor.

By interfaces, we can achieve multiple inheritance.

Since Java 8, interfaces can have both default and static methods.

This feature is used to achieve loose coupling.

Interface vs. Abstract Classes

Interface

- Can be implemented by a class using the keyword "implements"
- Can extend multiple interfaces
- Has only static and final variables
- Contains only abstract methods (However, it can have both static and default and static methods starting Java 8)
- Cannot provide the implementation of the abstract class
- Members of a Java interface are public by default

```
public interface Drawable{  
    void draw();  
}
```

Abstract Class

- Can be extended using the keyword "abstract"
- Can extend another Java class and implement multiple Java interfaces
- Can have abstract and non-abstract methods
- Can have final, non-final, static and non-static variables
- Can provide the implementation of interface
- Can have class members, such as private and protected

```
public abstract class Shape{  
    public abstract void draw();  
}
```

Knowledge Check 2

What is the return type of constructors?

A. int

B. float

C. void

D. None of the above

Which method can be defined only once in a program?

A. The main method

B. The finalize method

C. The static method

D. The private method

Which of these options can be overloaded?

A. Methods

B. Constructors

C. Both

D. None of the above

Which of these keywords is used to refer to the member of a base class from a subclass?

- A. Upper
- B. Super
- C. This
- D. None of the above

What should all variables of a class be ideally declared as?

- A. private
- B. public
- C. protected
- D. default

Which of these keywords is used to prevent content of a variable from being modified?

- A. final
- B. last
- C. constant
- D. static

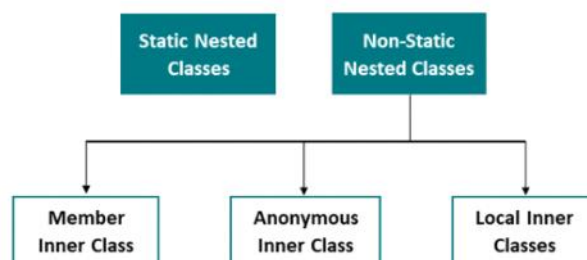
What Are Nested Classes?

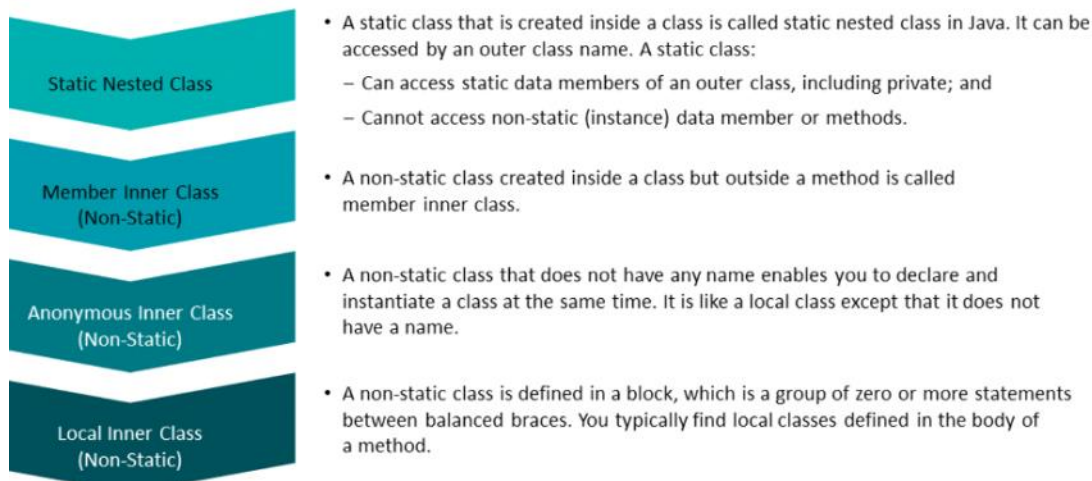
Is a class within another class or a logical grouping of classes that are only used in one place.

A nested class:

- Increases encapsulation
- Leads to more readable and maintainable code
- Non-static nested classes are also called inner classes

Types of Nested Classes





Arrays

- A Java array is an object that contains/stores elements of a similar data type in contiguous memory locations.
- A Java array can store only a fixed set of elements.
- The two types of arrays are:
 - Arrays
 - Multi-Dimensional Arrays

Arrays Excel At	Arrays Limitation
Code Optimization: – It optimizes the code, making the retrieval or sorting of data efficient. Random Access: – We can get any data located at an index position.	Size Limit: • We can store only a fixed number of elements in the array. • It doesn't grow its size at the runtime. • To solve this problem, a collection framework is used in Java which grows automatically.

Arrays (Cont.)

Advantages	Disadvantages
Efficient storage and access	Fixed size
Simple and easy to use	No built-in methods for manipulation
Performance	No automatic resizing
Memory efficiency	Homogeneous elements
Multidimensional support	Static type

Declaration and Instantiation

- **Array—Declaration:**

- To declare an array variable, you specify the data type of the elements followed by '[]' and then the variable name.
- Example : `int[] myArray;`

- **Array—Instantiation:**

- To initialize an array, you need to allocate memory for it using the new keyword, followed by the data type, and the size of the array in square brackets.
- Example : `int[] myArray = new int[5];` // Creates an array of size 5

- **Array—Declaration and Instantiation Together:**

- Can also declare and instantiate an array at the same time as follows.
- Example: `int[] myArray = new int[5];`

Initialization

Assigning Values:

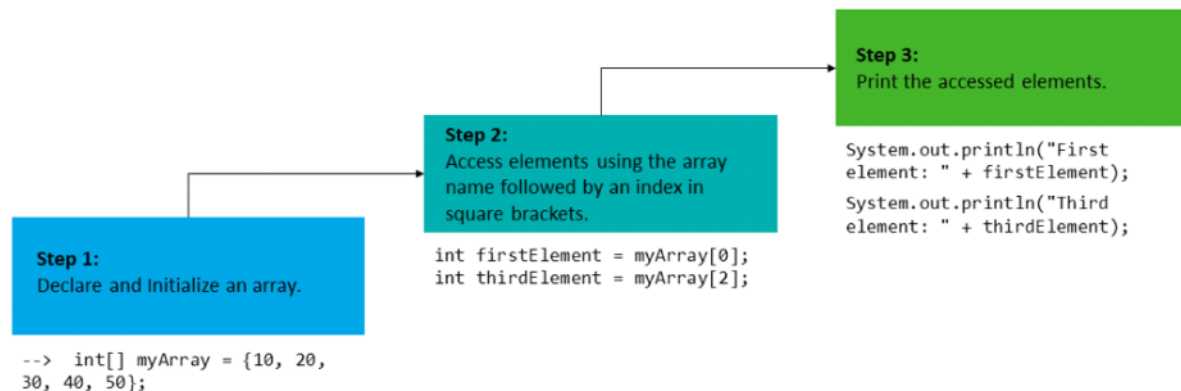
- Array is initialized by assigning values to the array elements.
- Initialization is done during declaration or later using the array index.
- Example:

```
// Declaration and Initialization by assigning values at same time
int[] myArray = {1, 2, 3, 4};
```

```
// Assigning values to the array later
myArray[0] = 1;
myArray[1] = 2;
myArray[2] = 3;
myArray[3] = 4;
```

Accessing Elements

- Each element in an array is assigned a unique index, which determines its position in the array.
- The indexing of an array starts from 0, meaning the first element of the array is at index 0, the second element is at index 1, and so on.



Note:

- Accessing an index that does not exist (an index > array length - 1) will throw `ArrayIndexOutOfBoundsException`.
- In the above example, accessing `myArray[5]` results in this exception as the array indices only go from 0 to 4.
- Array indexes—start from 0 and go up to the length of the array minus 1.
- So, the first element of the array is at index 0, and the last element is at index (array length - 1).

Array Length and Iterating Over Arrays

Array Length:

You can get the length of an array using the 'length' property.

```
int[] myArray = {10, 20, 30, 40, 50};  
int length = myArray.length; // Length is 5
```

Iterating over Arrays:

You can iterate over arrays using loops like 'for' or enhanced 'for' loop.

```
int[] myArray = {10, 20, 30, 40, 50};  
for (int i = 0; i < myArray.length; i++) {  
    System.out.println(myArray[i]);  
}  
// Or using an enhanced for loop  
for (int num : myArray) {  
    System.out.println(num);  
}
```

Multidimensional Arrays

Java supports multidimensional arrays, which are arrays of arrays.

Declaration and Instantiation	<pre>int[][] matrix = new int[3][3]; // 3x3 matrix</pre>
Initialization	<pre>matrix[0][0] = 1; // Assigning value 1 to row 0, column 0 element</pre>
Accessing Elements	<pre>int element = matrix[0][0]; // Accesses the element at row 0, column 0</pre>

Hands-On Activity 1

/>

Activity Details	
Problem Statement	Write a program that takes two inputs (integer/double/string) from the user and calls the method "add" (input1, input2). Create multiple methods (add) to accept any of the above datatype parameters. If both the inputs are numeric, then print the sum. If either of the inputs or both the inputs are of type string, then concatenate both the inputs and display.
Sample Input 1	5 4
Sample Input 2	1.1 2.2
Sample Input 3	Hello World

Hands-On Activity 2

'>

Activity Details	
Problem Statement	Create a class "Car" with private properties "company," "model," "engineVersion," and "carType." Create three objects and populate the private fields using constructors: • Car(company) • Car(company, carType) • Car(engineVersion, Cartype) Override the toString method and print the object.

Summary

Key learning points of the module:

- Java constructor chaining involves calling one constructor from another within the same class using `this()` keyword, allowing initialization of objects with different parameters while avoiding code duplication. It's useful for reusing initialization logic and maintaining code conciseness but be cautious to avoid infinite loops or unintended side effects.
- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- The main ideas behind Java's Object-Oriented Programming (OOP) concepts include abstraction, encapsulation, inheritance, and polymorphism.
- Arrays are efficient and simple data structures for storing collections of elements with fixed size and homogeneous data types. However, their lack of flexibility and dynamic resizing capabilities can be limiting in certain scenarios.

Object,String API,Wrappers,and Memory Management

Sunday, July 21, 2024 10:58 PM

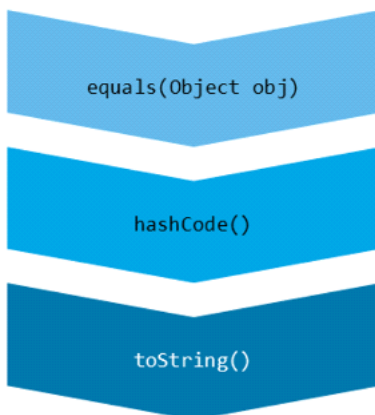
Topic	Descriptions
Object, String API and Wrapper Classes	Object Class and its Methods, String API, Wrapper Classes
Memory Management	Memory Collection in Java, Garbage Collection in Java

By the end of this session, you will be able to:

- Explain objects and use object methods
- Explain how strings may be used for representing a sequence of characters in the Java language
- Illustrate the use of string-related operations in Java programs
- Differentiate between String buffer and String builder
- Describe the concept of wrapper classes in Java
- Discuss Memory Management in Java

Object Class and its Methods

- In Java, the “Object” class is the root class for all Java classes.
- Every class in Java is a subclass of the “Object” class, either directly or indirectly.
- The “Object” class provides several methods that are available to all objects in Java.
- Here are some of the commonly used methods of the `Object` class:



- This method compares the current object with the specified object for equality.
- The default implementation in the “Object” class compares memory addresses, but it's often overridden in subclasses to provide custom equality checks.
- This method returns a hash code value for the object.
- It's used when storing objects in hash-based data structures like “HashMap”.
- It's recommended to override this method when the “equals” method is overridden.
- This method returns a string representation of the object.
- By default, it returns the class name followed by the memory address of the object.
- Subclasses often override this method to provide more meaningful string representations.

Object Class and its Methods (Cont.)

`getClass()`

- This method returns the runtime class of the object.
- It returns a "Class" object representing the class of the object.

`clone()`

- This method creates and returns a copy of the object.
- It's used to create a shallow copy of the object.
- To support cloning, the class must implement the "Cloneable" interface and override the "clone" method.

`finalize()`

- This method is called by the garbage collector before reclaiming the memory occupied by the object.
- Subclasses can override this method to perform cleanup operations before the object is garbage collected, but its use is discouraged in modern Java programming due to unpredictable behavior.

Object Class and its Methods (Cont.)

`wait(), notify(),
notifyAll()`

- These methods are used for inter-thread communication and synchronization.
- They are used in concurrent programming to coordinate the execution of threads.

- These are some of the key methods provided by the "Object" class in Java.
- Understanding these methods is essential for effective Java programming, as they are fundamental to the behavior of all objects in Java.

STRING API AND WRAPPER CLASSES

Strings and Their Operations

Strings

- Java strings are not primitive types.
- Java strings are objects and immutable.
- Two ways to create a string in Java:
 - Using a string literal (Example:
String name = "Java Language")
 - Using a new keyword (Example:
String name = new String ("Java Language")

Example:

Converting the case

- `name.toLowerCase()`
- `name.toUpperCase()`

Concatenating strings

- `String str = name + " is platform independent";`

Replacing parts of string

- `name.replace("Java", "Java Programming");`

Converting a string to array

- `name.getBytes()`
- `name.toCharArray()`

Converting primitives to strings

- `String.valueOf(123)`
- `String.valueOf(true)`

Parsing a string for primitive type

- `Integer.parseInt("123")`

String Buffer vs. String Builder

String Buffer and String Builder both are mutable in nature.

String Buffer

StringBuffer operations are thread-safe and synchronized, but lower in performance.

Example:

```
public class BufferTest{
    public static void main(String[]
    args){
        StringBuffer buffer=new
        StringBuffer("hello");
        buffer.append("world");
        System.out.println(buffer);
    }
}
```

String Buffer vs. String Builder

String Builder

StringBuilder operations are not thread-safe, but their performance is faster because there is no synchronization.

Example:

```
public class BuilderTest{
    public static void main(String[]
    args){
        StringBuilder builder=new
        StringBuilder("hello");
        builder.append("world");
        System.out.println(builder);
    }
}
```

Wrapper Classes/Autoboxing and Auto Unboxing

Wrapper classes

- Wrapper classes are used to convert Java primitives into reference types (objects).
- Since Java 5, the autoboxing and unboxing features automatically convert primitives into objects and objects into primitives.
- The automatic conversion of primitive into an object is known as autoboxing and vice-versa is auto unboxing.

Autoboxing and Auto Unboxing

- Constructing the object using a new operator is autoboxing.
- Extracting the value from the object is unboxing.

Example:

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Which method of class StringBuffer is used to concatenate the string representation to the end of invoking string?

concat()

B. append()

join()

D. concatenate()

Hands-On Activity 1

>

Activity Details	
Problem Statement	Write a program that prints if the given input string is a palindrome or not.
Sample Input 1	Malayalam
Sample Input 2	Hello

Hands-On Activity 2

>

Activity Details	
Problem Statement	Write a program to read a sentence and remove words that are used more than once, irrespective of the casing.
Sample Input	I can can I do this Do do this do do this this

Memory Allocation in Java

- Process of setting aside virtual memory sections in a program for storing the structures/classes instances and variables.
- However, it is just the reference that gets created at the time of declaration. There is no memory allocation that happens.
- The `new()` keyword is used to allocate heap memory to an object.
- The Java memory allocation is divided into the following sections for effective management:
 - **Heap:** Contains objects
 - **Stack:** Contains methods, local variables, and reference variables
 - **Code:** Contains your bytecode
 - **Static:** Contains static data/methods

Garbage Collection in Java

- Java programs perform automatic memory management through the process of garbage collection.
- Java programs that are compiled to bytecode can be run on a JVM for a short span of time.
- When running on the JVM, Java programs create objects on the heap, which dedicates a portion of memory to the program. Eventually, objects will not be needed.
- To free up memory, the garbage collector finds unused objects and deletes them.

Benefits of Java Garbage Collection

The garbage collector (part of the JVM) automatically removes unreferenced objects from the heap memory making Java more memory efficient.

Which of the following models is a garbage collection technique?

A. Cleanup Model

B. Mark and Sweep Model

C. Space Management Model

D. Sweep Model

Summary

Key learning points of the module:

- The `Object` class in Java serves as the superclass for all classes, providing fundamental methods like `equals`, `hashCode`, and `toString`. Subclasses inherit these methods and can override them to customize behavior such as equality comparisons and string representations.
- A wrapper class converts a primitive into an object and an object into a primitive. The automatic conversion of primitive into an object is known as autoboxing and vice-versa is unboxing.
- Process of setting aside virtual memory sections in a program for storing the structures/classes instances and variables.
- Java programs perform automatic memory management through the garbage collection process.

JAVA COLLECTION FRAMEWORK

Sunday, July 21, 2024 11:13 PM

Topic	Descriptions	Duration
Java Collection Framework	Collection Hierarchy, Advantages of Collection Framework, List, Set, Queue, Map, Iterator, ListIterator, Generics, Comparable, and Comparator	8 hours

Learning Objectives

By the end of this session, you will be able to:

- Explain the Java collections framework and its hierarchy
- Describe generics and their benefits
- Discuss collection framework algorithms
- Analyze and make use of appropriate collection(s) and related algorithm(s) to group, store, retrieve, and manipulate multiple elements/objects into a single unit in Java programs/applications

Collections framework , generics

What Are Collections?

- A collection, which is also known as a container, is simply an object that groups multiple elements into a single unit.
- Collections are used to store, retrieve, manipulate, and communicate aggregate data.
- Typically, they represent data items that form a natural group. Examples of collections include a poker hand (a collection of cards), a mail folder (a collection of letters), or a telephone directory (a mapping of names to phone numbers).

Collection Framework Overview

A collection framework is a unified architecture for representing and manipulating objects. It contains the following parts:

Interfaces:

- These are the abstract data types to represent the collection.
- `java.util.Collection` is the root interface of the Java collection framework.
- Examples include list, set, queue, and map.

Implementation Classes:

- These are the concrete implementation classes for collections.
- Some important Java collection classes are `ArrayList`, `LinkedList`, `HashMap`, `TreeMap`, `HashSet`, and `TreeSet`.

Algorithms:

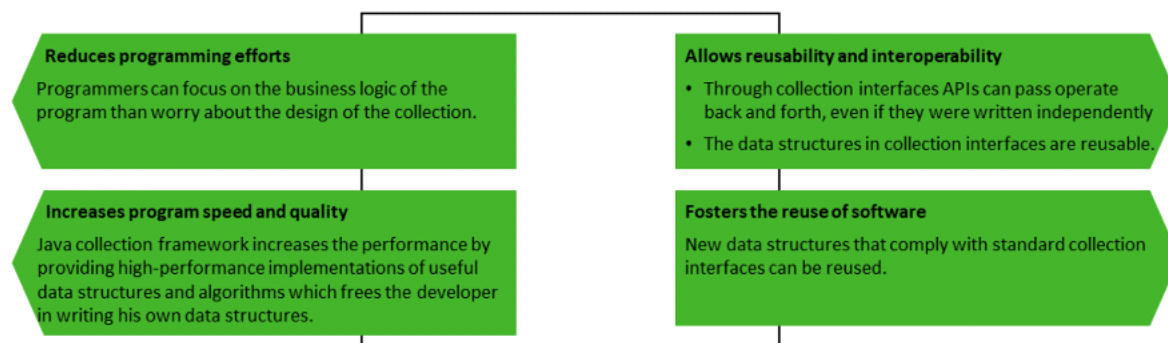
- These are the methods that perform useful computations, such as searching, sorting, and shuffling.

Java Collection Framework

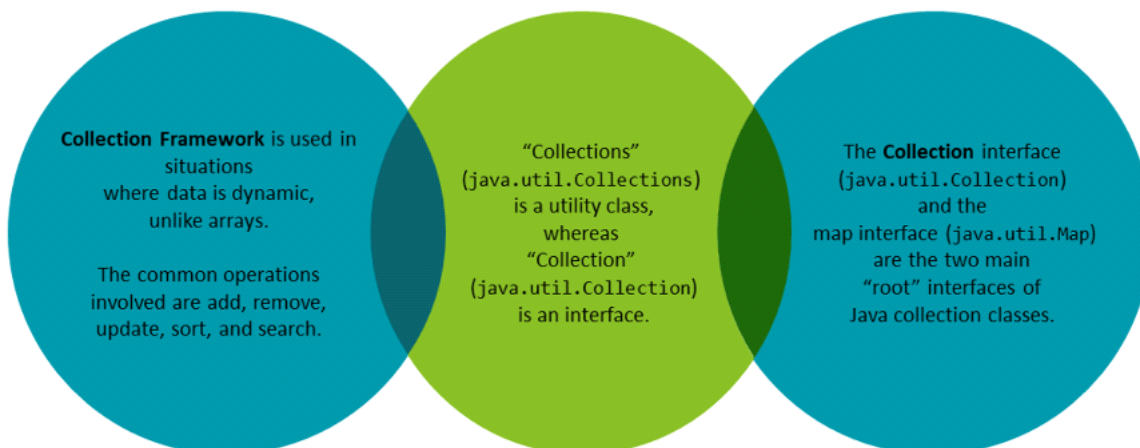


- Java Collection API provides a set of interfaces and classes to store data dynamically.
- It also allows quick data manipulation as compared to an array.

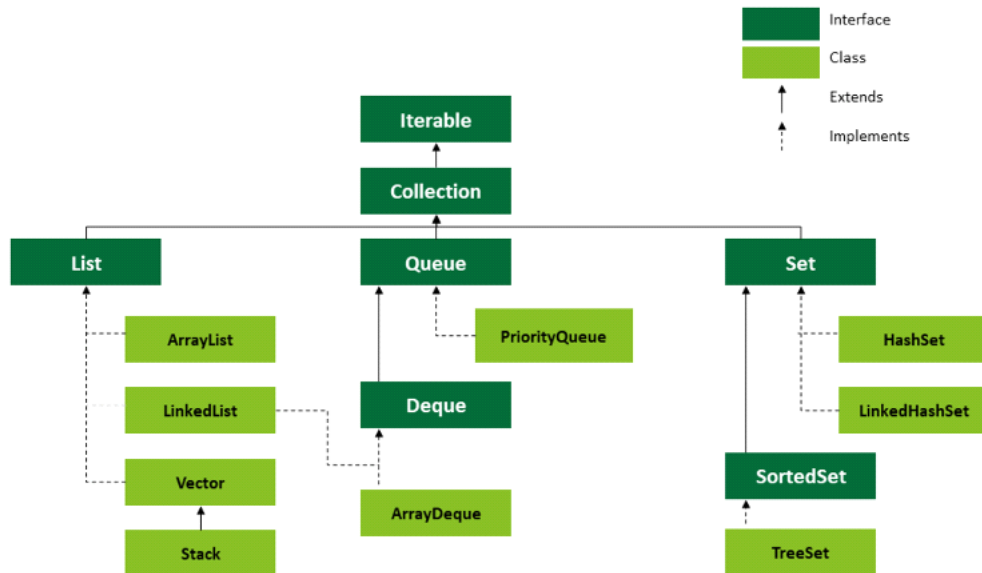
Benefits of Java Collection Framework



Collections vs. Collection



Collection Hierarchy



Which of these packages contains all the collection classes?

A. `java.lang`

B. `java.util`

C. `java.net`

D. `java.awt`

Which of these interfaces is NOT part of the Java Collection Framework?

A. List

B. SortedList

C. SortedMap

D. Set

What does collection refer to in Java?

A. A group of objects

B. A group of interfaces

C. A group of classes

D. None of the above

List

Definition:

- The List interface is a core part of Java's Collection Framework.
- It is a child interface of the Collection interface and implements the Iterable interface.
- The List interface provides a way to store the ordered collection.

Key Characteristics

- **Order:**
 - Lists in Java maintain the order of elements based on the sequence in which they were added.
- **Index-based Methods:**
 - Lists provide index-based methods to insert, update, delete, and search the elements.
- **Duplicates:**
 - Lists may contain duplicate elements.
- **Nulls:**
 - Lists in Java allow null elements.

Sub-interfaces and Implementing Classes

- There are two main implementing classes of the List interface—ArrayList and LinkedList.
- ArrayList and LinkedList are widely used in Java programming due to their functionality and flexibility.

Usage

- Lists are commonly used data structures in Java. They can be used to store an ordered collection of items, which may include duplicates.
- They are particularly useful when you need constant-time access to individual elements in your collection, as they support positional access and insertion of elements.

Types of Lists

ArrayList	Linked List	Vector
- ArrayList is the implementation of a List Interface, where the elements can be dynamically added or removed from the list. - The size of the list increases dynamically—it increases if more elements are added than the initial size. - It is not thread-safe. - Syntax: <code>ArrayList object = new ArrayList ();</code>	- A linked list is a sequence of links containing items. - Each link contains a connection to another link. - Java Linked List classes use two types of lists to store elements: - Singly Linked List - Doubly Linked List - It is an ideal choice to implement stacks—Last In First Out (LIFO) or queues—First In First Out (FIFO). - Syntax: <code>LinkedList object = new LinkedList();</code>	- Vectors are like arrays, where the elements of the vector object can be accessed via an index into the vector. - A Vector is the same as an ArrayList except with all its methods synchronized, where synchronization enables us to prevent multiple threads accessing the same instance at the same time. - Syntax: <code>Vector object = new Vector(size, increment)</code>

Array List, Linked List, and Vector—Illustration

Array List

```
import java.util.*;
public class ArrayListExample {
    public static void main(String
args[]) {
        List<Integer> marks = new
ArrayList<> ();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
    }
}
```

Linked List

```
import java.util.*;
public class LinkedListExample {
    public static void main(String
args[]) {
        List<Integer> marks = new
LinkedList<> ();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
    }
}
```

Vector

```
import java.util.*;
public class VectorExample {
    public static void main(String
args[]) {
        List<Integer> marks = new
Vector<> ();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
    }
}
```

LIST API AND ITERATORS

List API and Iterators

Index-Based Methods	• The List interface provides a variety of methods to insert, update, delete, and search elements based on their index. • The index of elements in a List starts from 0 (just like an array). • Some of the important index-based methods are: – add(int index, E element) – get(int index) – set(int index, E element) – remove(int index)
Iterator	• Iterator interface is a tool for traversing through collections in Java, such as List. • It is a Universal Iterator as it can be applied to any Collection object. • It can pass the Iterator object as a value parameter to a method and then use the Iterator methods to access the elements in the collection. • It has methods like hasNext(), next(), and remove().
List Iterator	• The List interface provides a special iterator, the ListIterator, that allows element iteration in both the forward and backward directions. • The listIterator() method returns a ListIterator object which can be used to traverse the List in either direction.

Queue

Definition:

- It is used to hold the elements about to be processed in FIFO (First In First Out) order.
- It is an ordered list of objects with its use limited to insert elements at the end of the list and deleting elements from the start of the list, (i.e., it follows the FIFO or the First-In-First-Out principle).

Key Characteristics

- **FIFO (First-In-First-Out):**
 - Queues in Java are designed to follow the FIFO (First-In-First-Out) concept meaning that the element inserted first will be removed first.
- **Element Insertion:**
 - Elements are inserted at the tail using the add() or offer() methods.
- **Element Removal:**
 - Elements are removed from the head using the remove() or poll() methods.
- **Element Retrieval:**
 - element() or peek() methods are used to retrieve the head of the Queue without removing it.

Queue (Cont.)

Sub-interfaces and Implementing Classes

Some of the classes that implement the Queue interface include LinkedList, PriorityQueue, and ArrayDeque.

Usage

- Queues are often used in multi-threaded programs to synchronize tasks.
- In order processing and similar scenarios, as orders come in, get added to the Queue and processed in the arrival order.
- In resource management, for example, in a print spooler where multiple print requests are sent concurrently, a queue can manage these requests in the order they were received.

Stack vs. Queue

- A **stack** follows the LIFO principle, which means the element inserted at the last is the first element to come out.
- The stack is the subclass of Vector, and supports methods like push(), pop(), where:
 - Insert operation is called push; and
 - Delete operation is called pop.
- In stack we always keep track of the last element present in the list with a pointer called top.
- A stack is simpler than a queue.

Stack vs. Queue

- A **queue** interface maintains the FIFO order.
 - Supports methods like peek(), poll() and remove()
- The following classes implement the queue interface:
 - PriorityQueue class: Holds the elements or objects which are to be processed by their priorities; and implements the queue interface
 - Deque: Stands for a double-ended queue which enables us to perform the operations at both the ends like the deck (as it is pronounced) of cards; and extends the queue interface
 - ArrayDeque: Implements the deque interface. Unlike queue, we can add or delete the elements from both ends

Stack, Queue, and Deque—Illustration

/>

Stack

```
import java.util.*;
public class StackExample {
    public static void main(String
args[]) {
        Stack<Integer> marks = new
Stack<> ();
        marks.push (56);
        marks.push (62);
        marks.push (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
        System.out.println("Popping the
elements");
        while (!marks.isEmpty())
            System.out.println(marks.pop());
    }
}
```

Queue

```
import java.util.*;
public class QueueExample {

    public static void main(String args[]) {
        Queue<Integer> marks = new
PriorityQueue<>();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
        System.out.println("Polling the
elements");
        while (!marks.isEmpty())
            System.out.println(marks.poll());
    }
}
```

Deque

```
import java.util.*;
public class DequeExample {

    public static void main(String
args[]) {
        Deque<String> deque = new
ArrayDeque<String>();
        deque.add ("abc");
        deque.add ("def");
        deque.add ("ghi");
        deque.addFirst ("First
element");
        deque.addLast ("Last");

        System.out.println("Traversal
using for");
        for(String str: deque)
            System.out.println(str);
    }
}
```

Set

Definition:

- The Set interface in Java represents a collection that does not contain duplicate elements.
- This interface models the mathematical set abstraction and is used to manipulate sets of elements.
- It extends the Collection interface.

Key Characteristics

- **No Duplicate Elements:**
 - Set does not support storing duplicate values.
 - This is the primary distinction between a Set and other forms of the Collection like List and Map.
- **One Null Element:**
 - In a Set, null element can be stored, but you can insert it only once.
- **Unordered and Unindexed:**
 - A Set in Java does not maintain insertion order and also not indexed. So, they do not offer positional access.

Set (Cont.)

Implementing Classes

The Set interface has various Implementation classes available like HashSet, LinkedHashSet, EnumSet, TreeSet , etc.,

Usage

- It is used for storing a set of unique values like IDs or usernames where each ID or username must not be repeated.
- It supports mathematical set operations like union, intersection, and subset, where you might have a set of employees in one department and another set in a different department, and you want to find employees who are in both (intersection), or all employees (union), etc.

Set - Implementations

/>

Types of Set Implementation



HashSet

- HashSet only contains unique elements.
- It inherits the AbstractSet class and implements the set interface.
- It uses a mechanism known as hashing to store the elements.



TreeSet

- Java TreeSet class implements the set interface which uses a tree for storage.
- The objects of this class are stored in ascending order and use binary search.
- It inherits AbstractSet class and implements the NavigableSet interface.
- It contains unique elements such as HashSet.



LinkedHashSet

- Java LinkedHashSet class is a hash table and linked list implementation of the set interface.
- It contains unique elements such as HashSet and maintains the insertion order.

HashSet, LinkedHashSet, and TreeSet—Illustration

</>

HashSet

```
import java.util.*;
public class HashSetDemo {
    public static void main(String
args[]) {
    // Creating HashSet and adding
elements
    HashSet<String> hs = new
HashSet<String>();
    hs.add("Java");
    hs.add("Collections");
    hs.add("Is");
    hs.add("Very");
    hs.add("Helpful");
    // Traversing elements
    Iterator<String> itr =
hs.iterator();
    while (itr.hasNext())
        System.out.println(itr.next());
    }
}
```

LinkedHashSet

```
import java.util.*;
public class LinkedHashSetDemo {
    public static void main(String
args[]) {
    // Creating LinkedHashSet and adding
elements
    HashSet<String> lhs = new
LinkedHashSet<String>();
    lhs.add("Java");
    lhs.add("Collections");
    lhs.add("Is");
    lhs.add("Very");
    lhs.add("Helpful");
    // Traversing elements
    Iterator<String> itr =
lhs.iterator();
    while (itr.hasNext())
        System.out.println(itr.next());
    }
}
```

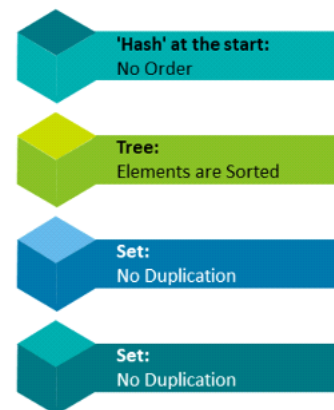
TreeSet

```
import java.util.*;
public class TreeSetDemo {
    public static void main(String
args[]) {
    // Creating TreeSet and adding
elements
    Set<String> ts = new
TreeSet<String>();
    ts.add("Java");
    ts.add("Collections");
    ts.add("Is");
    ts.add("Very");
    ts.add("Helpful");
    // Traversing elements
    Iterator<String> itr =
ts.iterator();
    while (itr.hasNext())
        System.out.println(itr.next());
    }
}
```

Do you Know?

- **HashSet** implements Set (no duplicates, no order).
- **LinkedHashSet** implements Set (no duplicates but ordered).
- **TreeSet** implements NavigableSet (no duplicates, sorted in the natural order).

Set	Allows unique values but no duplication
List	Cares about which position each object is in
Queue	Arranges in the order of processing (for example, to-do list)



MAP

Map

>

Definition:

- Map interface in the Java Collections Framework represents a mapping between a set of keys and a set of values.
- Each key is linked to a specific value.
- Once stored in a Map, you can access a value using its corresponding key.

Key Characteristics

- **Key-Value Pairs:**
 - Map stores data in key-value pairs, also known as entries.
 - Each entry consists of a unique key and a corresponding value associated with that key.
- **Unique Keys:**
 - In a Map, each key must be unique. However, values can be duplicated.
- **Null Keys and Values:**
 - Depending on the specific implementation, a Map may allow one null key and multiple null values.
- **No Intrinsic Order:**
 - Maps don't guarantee any specific order of entries (except few sub-classes).

Map (Cont.)

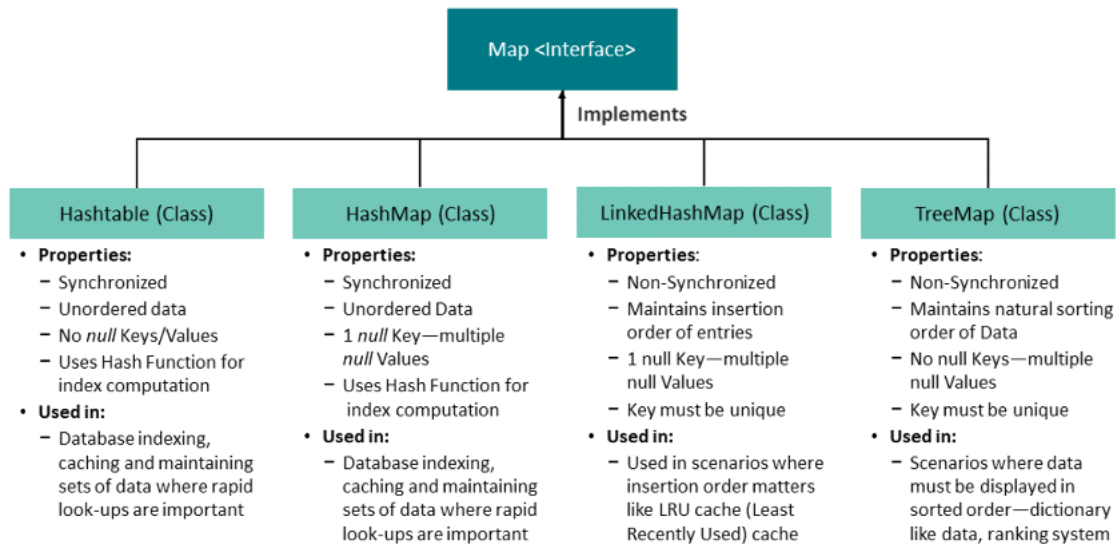
Implementing Classes

Classes that implement the Map interface include HashMap, TreeMap, and LinkedHashMap.

Usage

- Map is used to create dictionary-like data structures, where keys are words and values are their definitions.
- For quick Data Searches, e.g., a database of students where the key is student id and value is the student object.
- Map is used to count the frequency of elements in a collection.
- For instance, in a list of names, you could use a Map to store the frequency of each name where keys are the names and values are their frequencies.

Map—Implementations (Diagrammatic View)



HashMap and Hashtable—Illustration

/>

HashMap

```
import java.util.*;
public class HashMapDemo {
    public static void main(String args[]) {
        // Creating HashMap and adding elements
        Map<Integer, String> hm = new
HashMap<>();
        hm.put(1, "Java");
        hm.put(2, "Servlets");
        hm.put(3, "JSP");
        // Finding the value of a key
        System.out.println("Value for 1 is " +
hm.get(1));
        // Traversing through the HashMap
        for (Map.Entry<Integer, String> e :
hm.entrySet())
            System.out.println(e.getKey() + " " +
e.getValue());
    }
}
```

Hashtable

```
import java.util.*;
public class HashtableDemo {
    public static void main(String args[]) {
        // Creating Hashtable and adding elements
        Map<Integer, String> ht = new
Hashtable<>();
        ht.put(1, "Java");
        ht.put(2, "Servlets");
        ht.put(3, "JSP");
        // Finding the value of a key
        System.out.println("Value for 1 is " +
ht.get(1));
        // Traversing through the Hashtable
        for (Map.Entry<Integer, String> e :
ht.entrySet())
            System.out.println(e.getKey() + " " +
e.getValue());
    }
}
```

HashMap vs. Hashtable

HashMap

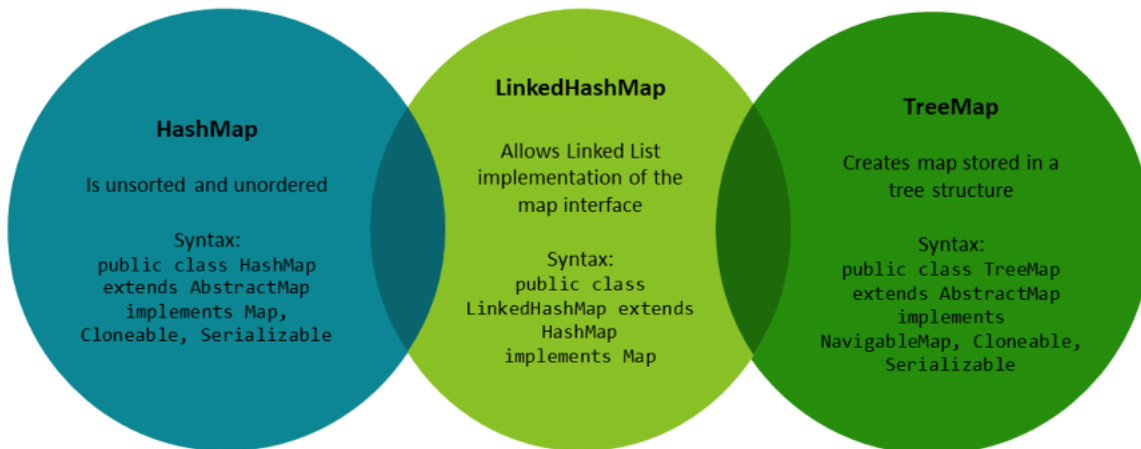
- Is non-synchronized, which means it is not thread-safe
- Allows only one null key and multiple null values
- Is fast
- Is traversed by Iterator
- Inherits AbstractMap class

Hashtable

- Is synchronized and can be shared with many threads
- Does not allow null key or its value
- Is slow
- Is traversed by enumerator and iterator
- Inherits the Dictionary class

HashMap is generally preferred over Hashtable, if thread synchronization is not needed.

HashMap vs. LinkedHashMap vs. TreeMap



HashMap vs LinkedHashMap vs TreeMap—Illustration

</>

HashMap

```
import java.util.*;
class HashMapDemo
{
    // This function prints ordering of
    // all elements
    static void
    insertAndPrint(AbstractMap<Integer,
    String> map)
    {
        int[] array= {1, -1, 0, 2,-2};
        for (int x: array)
            map.put(x,
            Integer.toString(x));
        for (int k: map.keySet())
            System.out.print(k + ", ");
    }
    // Driver method to test above method
    public static void main (String[]
    args)
    {
        HashMap<Integer, String> map =
        new HashMap<Integer, String>();
        insertAndPrint(map);
    }
}
```

LinkedHashMap

```
import java.util.*;
class LinkedHashMapDemo
{
    // This function prints ordering of
    // all elements
    static void
    insertAndPrint(AbstractMap<Integer,
    String> map)
    {
        int[] array= {1, -1, 0, 2,-2};
        for (int x: array)
            map.put(x,
            Integer.toString(x));
        for (int k: map.keySet())
            System.out.print(k + ", ");
    }
    // Driver method to test above method
    public static void main (String[]
    args)
    {
        LinkedHashMap<Integer, String>
        map = new LinkedHashMap<Integer,
        String>();
        insertAndPrint(map);
    }
}
```

TreeMap

```
import java.util.*;
class TreeMapDemo
{
    // This function prints ordering of
    // all elements
    static void
    insertAndPrint(AbstractMap<Integer,
    String> map)
    {
        int[] array= {1, -1, 0, 2,-2};
        for (int x: array)
            map.put(x,
            Integer.toString(x));
        for (int k: map.keySet())
            System.out.print(k + ", ");
    }
    // Driver method to test above method
    public static void main (String[]
    args)
    {
        TreeMap<Integer, String> map =
        new TreeMap<Integer, String>();
        insertAndPrint(map);
    }
}
```

List vs. Set vs. Queue vs. Map

Property	List	Set	Queue	Map
Ordering	Maintains insertion order	Does not maintain any order	Maintains order (FIFO or priority based)	Does not maintain any order (except some implementations like LinkedHashMap or TreeMap)
Duplicate Elements	Allows duplicates	Does not allow duplicates	Allows duplicates	Does not allow duplicate keys, allows duplicate values
Null Elements	Allows null elements	Allows null elements (only once in a Set)	Allows null elements	Allows null values, and one null key
Data Structure	Linear data structure	Mathematical set	Linear data structure	Key-Value pairs
Use Case	When order matters and duplicates are allowed	When you want to store unique elements	When elements need to be processed in a particular order (FIFO or priority)	When you want to map keys to values
Main Implementations	ArrayList, LinkedList	HashSet, TreeSet	PriorityQueue, ArrayDeque	HashMap, TreeMap

Iterator and Iterable Interfaces

Iterator Interface (java.util.iterator)

- An interface that iterates the elements
- Used to modify the elements and traverse the list
- Has three methods:
 - **public boolean hasNext()**—Returns true if the iterator has more elements
 - **public Object next()**—Returns the element and moves the cursor pointer to the next element
 - **public void remove()**—Removes the last elements returned by the iterator
- The Iterator interface is also known as Universal Java Cursor as it is applicable for all the classes of the collection framework.

Iterable Interface (java.lang.iterable)

- The Iterable interface provides only one method which returns an iterator.
- The Iterator iterator() returns an iterator.

Iterator vs. ListIterator Interfaces

/>

Both Iterator and ListIterator are utilized for traversing or iterating over the elements of collection classes.

Property	Iterator	ListIterator
Type	Interface	Interface and it extends 'Iterator'
Direction of Traversal	Can only traverse in forward direction	Can traverse in both forward and backward directions
Add Elements	Does not have the ability to add elements to the collection	Can add elements to the collection while traversing
Replace Elements	Does not have the ability to replace elements in the collection	Can replace the last element returned by next or previous methods
Indexing	Does not support indexing	Supports indexing—can return the index of the element that would be returned by a subsequent call to the next or previous
Usage	Can be used with all Collection Framework classes	Can only be used with List and its implementations
Cursor Movement	Only one way cursor movement	Two-way cursor movement (forward and backward)
Methods	Supports with methods like 'hasNext()' and 'next()'	Provides additional methods like 'hasPrevious()', 'previous()', 'add()', 'set()', and 'remove()'

Iterator vs. ListIterator Example

```
public class IteratorExample {
    public static void main(String[] args) {
        // Creating a list
        List<String> myList = new ArrayList<>();
        myList.add("Java");
        myList.add("Python");
        myList.add("JavaScript");

        // Using Iterator
        Iterator<String> iterator = myList.iterator();
        while (iterator.hasNext()) {
            String language = iterator.next();
            System.out.println(language);
        }
    }
}
```

```
// Using ListIterator for bidirectional traversal
ListIterator<String> listIterator =
myList.listIterator(myList.size());
System.out.println("Printing list in reverse:");
while (listIterator.hasPrevious()) {
    String language = listIterator.previous();
    System.out.println(language);
}
}
```

Which of these interfaces must contain a unique element?

A. Set

B. List

C. Array

D. Collection

Which of these methods can convert an object into a list?

A. SetList()

B. ConvertList()

C. SingletonList()

D. CopyList()

A map implements the collection interface.

Yes

No

What should you use to sort elements of ArrayList?

A. `Collection.sort(listObj);`

B. `listObj.sort();`

C. `Collections.sort(listObj);`

D. None of the above

What is the return type of the hasNext() method of an iterator?

A. Integer

B. Double

C. Boolean

D. Collections Object

Is the PriorityQueue thread-safe?

Yes

No

Which of these interfaces must contain a unique element?

A. Set

B. List

C. Array

D. Collection

Which of these methods can convert an object into a list?

A. SetList()

B. ConvertList()

C. SingletonList()

D. CopyList()

Which of these interfaces handle sequences?

A. Set

B. Comparator

C. List

D. Collection

Which of these options is a static variable defined in collections?

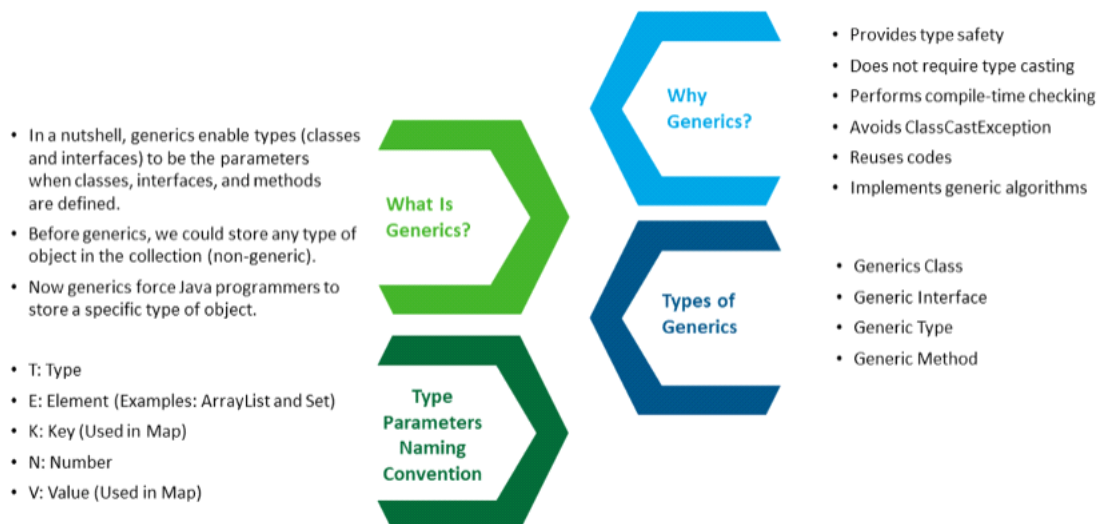
A. EMPTY_SET

B. EMPTY_LIST

C. EMPTY_MAP

D. All of the above

Generics—Overview



Generics—Benefits



Benefits of Generics

Provides type safety

- Generics force type safety in the java language.
- We can hold only a single type of object in generics. It does not allow the storage of other objects.
- Without generics, we can store any type of objects.

Does not require type casting

- There is no need to type cast the object. Before generics, we need to type cast.
- If we already know that our list only holds string data, then we need not type cast it every time.

Performs compile-time checking

- Generics make errors appear at the compile time instead of the run time.
- Generics add stability to your code by making more of your bugs detectable at the compile time.
- Generics were added in Java 5 to provide compile-time type checking and remove the risk of ClassCastException that was common with collection classes.

Generics Using ArrayList and Map—Illustration

</>

Generics Using ArrayList

```
import java.util.*;
class GenericsUsingArrayList {
    public static void main(String args[]) {
        ArrayList<String> list=new ArrayList<String>();
        list.add("abc");//works fine
        list.add("def");
        //list.add(32);//compile time error
        String s=list.get(1);//type casting is not
        required
        System.out.println("element is: "+s);
        Iterator<String> itr=list.iterator();
        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Generics Using Map

```
import java.util.*;
class GenericsUsingMap {
    public static void main(String args[]) {
        Map<Integer,String> map=new HashMap<Integer,String>();
        map.put(1,"abc");
        map.put(4,"def");
        map.put(2,"gji");
        //Now use Map.Entry for Set and Iterator
        Set<Map.Entry<Integer,String>> set=map.entrySet();
        Iterator<Map.Entry<Integer,String>>
        itr=set.iterator();
        while(itr.hasNext()){
            Map.Entry e=itr.next();//no need to typecast
            System.out.println(e.getKey()+" "+e.getValue());
        }
    }
}
```

Generic Class—Illustration

>

Java Generic Class

```
// We use < > to specify Parameter type
class Test<T> {
    // An object of type T is declared
    T obj;
    Test(T obj) {
        this.obj = obj; } // constructor
    public T getObject() {
        return this.obj;
    }
}

class GenericsClassExample1 {
    public static void main (String[] args) {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(25);
        System.out.println(iObj.getObject());
        // instance of String type
        Test<String> sObj = new Test<String>("Java Generics");
        System.out.println(sObj.getObject());
    }
}
```

Java Generic Class (Multiple Type Parameters)

```
// We use < > to specify Parameter type
class Test<T, U> {
    T obj1; // An object of type T
    U obj2; // An object of type U
    Test(T obj1, U obj2) {
        this.obj1 = obj1; // constructor
        this.obj2 = obj2; }
    // To print objects of T and U
    public void print() {
        System.out.println(obj1);
        System.out.println(obj2);
    }
}

class GenericsClassExample2 {
    public static void main (String[] args) {
        Test<String, Integer> obj = new Test<String,
        Integer>("Generics", 3);
        obj.print();
    }
}
```

Generic Method and Generic Interface—Illustration

>

Java Generic Method—Example

```
public class GenericsMethodExample {

    public static < E > void printArray(E[] elements) {
        for ( E element : elements){
            System.out.println(element );
        }
        System.out.println();
    }

    public static void main( String args[] ) {
        Integer[] intArray = { 10, 15, 20, 25, 30 };
        Character[] charArray = {'A','B','C'};
        System.out.println( "Printing Integer Array"
    );
        printArray( intArray );
        System.out.println( "Printing Character Array"
    );
        printArray( charArray );
    }
}
```

Java Generic Interface

```
"Comparable interface is a great example of Generics
in interfaces"

package java.lang;
import java.util.*;

public interface Comparable<T> {

    public int compareTo(T o);
}
```

Comparable vs. Comparator

/>

Comparable Interface

- Provides a single sorting sequence, which means it can be sorted based on a single element such as ID or name
- Compares an object with other objects of the same type
- Is used to sort the objects with a natural ordering
- Compares "this" reference with the object specified
- Affects the original class, which means that the actual class is modified
- Provides only one method to sort elements:
 - `int compareTo()`
- Is present in the `java.lang` package
- Can sort the list elements of the comparable type by using the `Collections.sort(List)` method

`java.lang.Comparable`
and
`java.util.Comparator`
are powerful interfaces
that can be used to sort
the collection elements.

Comparator Interface

- Provides multiple sorting sequences, which means it can be sorted based on multiple elements
- Is used to order the objects of a specific class
- Is used to sort attributes of different objects
- Compares two different class objects provided
- Doesn't affect the original class, which means that the actual class is not modified
- Provides two methods to sort elements:
 - `compare(Object obj1, Object obj2)`
 - `equals(Object element)`
- Is present in the `java.util` package
- Can sort the list elements of Comparator type by the `Collections.sort(List, Comparator)` method

Implementation of Comparable and Comparator Interfaces

```
// Example class representing a book
class Book implements Comparable<Book> {
    private String title;
    private int pageCount;

    public Book(String title, int pageCount) {
        this.title = title;
        this.pageCount = pageCount;
    }

    public String getTitle() {
        return title;
    }

    public int getPageCount() {
        return pageCount;
    }
}
```

```
// Implementing compareTo method for natural ordering
@Override
public int compareTo(Book other) {
    return this.pageCount - other.pageCount;
}

@Override
public String toString() {
    return "Book{" +
        "title='" + title + '\'' +
        ", pageCount=" + pageCount +
        '}';
}
```

Implementation of Comparable and Comparator Interfaces (Cont.)

```
public class Main {
    public static void main(String[] args) {
        // Creating a list of books
        List<Book> books = new ArrayList<>();
        books.add(new Book("Java in Action", 500));
        books.add(new Book("Introduction to Algorithms", 1000));
        books.add(new Book("Clean Code", 400));

        // Sorting books using Comparable (natural ordering)
        Collections.sort(books);
        System.out.println("Sorted books by page count (natural ordering):");
        for (Book book : books) {
            System.out.println(book);
        }

        // Sorting books using Comparator (title)
        Collections.sort(books, new TitleComparator());
        System.out.println("\nSorted books by title:");
        for (Book book : books) {
            System.out.println(book);
        }
    }
}
```

```
// Comparator implementation for sorting books by title
class TitleComparator implements Comparator<Book> {
    @Override
    public int compare(Book book1, Book book2) {
        return
            book1.getTitle().compareTo(book2.getTitle());
    }
}
```

Knowledge Check 14

Which of the following methods cannot be type parameterized?

A. Overloaded Methods

B. Generic Methods

C. Class Methods

D. Overriding Methods

Which of these type parameters is used for a generic method to return and accept any type of object?

A. K

B. N

C. T

D. W

Which of these data types cannot be parameterized?

A. Array

B. List

C. Map

D. Set

Which of these types cannot be used to initiate a generic type?

A. Integer Classes

B. Primitive Types

C. Float Classes

D. Collections

Which of these options is a wildcard symbol?

A. ?

B. !

C. %

D. &

Which of these keywords is used to upper bound a wildcard?

A. stop

B. bound

C. extends

D. implements

Which of the following keywords is used for lower bounding a wildcard?

A. Extends

B. Super

C. Lower

D. Class

What is the use of wildcards?

A. It is used in cases when the type that is being operated upon is not known.

B. It is used to make codes more readable.

C. It is used to access members of super classes.

D. It is used for type argument of the generic method.

Activity Details	
Problem Statement	Write a code for a map that associates the employees with the department. Let the keys of maps be an employee object and the associated value be a string.
Sample Input	N/A

Summary

Here are the key learning points of the module:

- The Java Collections Framework simplifies the management and manipulation of data structures in Java, offering a wide range of classes and interfaces for different collection types and providing algorithms and utilities for common collection operations. It is an essential part of Java programming and widely used in various applications and libraries
- The hierarchy of the entire collection framework consists of four core interfaces, such as Collection, List, Set, Map, and specialized interfaces.
- Collection: The root interface that defines the common behavior and operations for all collection types.
- List: An ordered collection that allows duplicate elements. Elements can be accessed by their index.
- Set: A collection that does not allow duplicate elements. There is no defined order for elements.
- Queue: A collection designed for holding elements prior to processing. It follows the FIFO (First-In-First-Out) principle.
- Map: An object that maps keys to values. Each key is unique, and the values can be retrieved using the associated key.
- if you need to iterate over elements in a list and require bidirectional traversal or the ability to modify the list while iterating, 'ListIterator' would be the better choice. Otherwise, if you just need to iterate forward over any collection, 'Iterator' suffices.

Summary (Cont.)

Here are the key learning points of the module (cont.):

- Generics enable types (classes and interfaces) to be the parameters when classes, interfaces, and methods are defined. It has several benefits, which include providing type safety, doesn't require casting and performing compile time checking.
- Thread Safety: The Java Collections Framework provides several synchronized implementations of collection classes, allowing multiple threads to safely access and modify collections concurrently.

GIT BASICS

Monday, July 22, 2024

10:35 PM

Learning Objectives

By the end of this session, you will have gained the ability to:

- Understand Git's role as a Version Control System (VCS)
- Master the terminology associated with Git
- Comprehend and apply the Git Workflow
- Grasp the basics of Git, including installation and command line usage
- Manage files effectively within Git
- Understand and utilize key Git processes
- Become familiar with Git Clients, with a particular emphasis on GitHub and its functionality.

What is Version Control?



Version control helps keep track of our work and helps easily explore the changes made to data, coding scripts, notes, etc.



Saving multiple files after each change like `main_script_1` or `main_script_v1` or `mainScript_Final`, etc., are types of version control.



Tens or hundreds of similar files will be left when you use the version control, making it cumbersome to directly compare different versions and share with the collaborators.



Version control becomes much smoother and easier to implement with help from software like GIT.

What is GIT?



Git is an open-source version control system or protocol used to track changes in files



Distributed version control systems like GIT give flexibility for version control, sharing, collaboration, and managing code in a consistent manner.

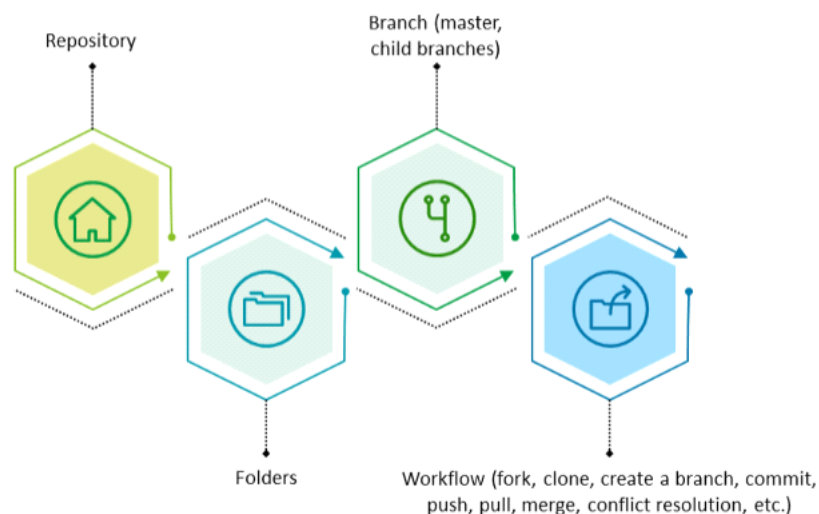


It allows collaboration among multiple developers working on the same project. Team members can publish, share, review, and iterate on code changes through GIT branches shared with others.

Key Concepts of GIT

- **Version Control:** Git allows you to keep track of the changes made to your code over time. Each change is recorded, along with who made the change and when.
- **Distributed:** Unlike centralized version control systems, each developer's working copy of the code is also a repository that can contain the full history of changes.
- **Snapshots:** Git thinks of data more like a series of snapshots of a miniature filesystem. With Git, every time you commit, or save the state of your project, Git basically takes a picture of what all your files look like at that moment and stores a reference to that snapshot.
- **Branching:** Branches in Git are lightweight and easy to create. They allow you to work on different features or bug fixes independently without affecting the main codebase.
- **Merging:** Git allows you to merge changes from one branch into another. This is useful when you have finished working on a feature branch and want to incorporate those changes into the main branch.

Basic Git Workflow



Git Workflow (Contd..)

1. **Initializing a Repository:** You start by creating a new Git repository or cloning an existing one.

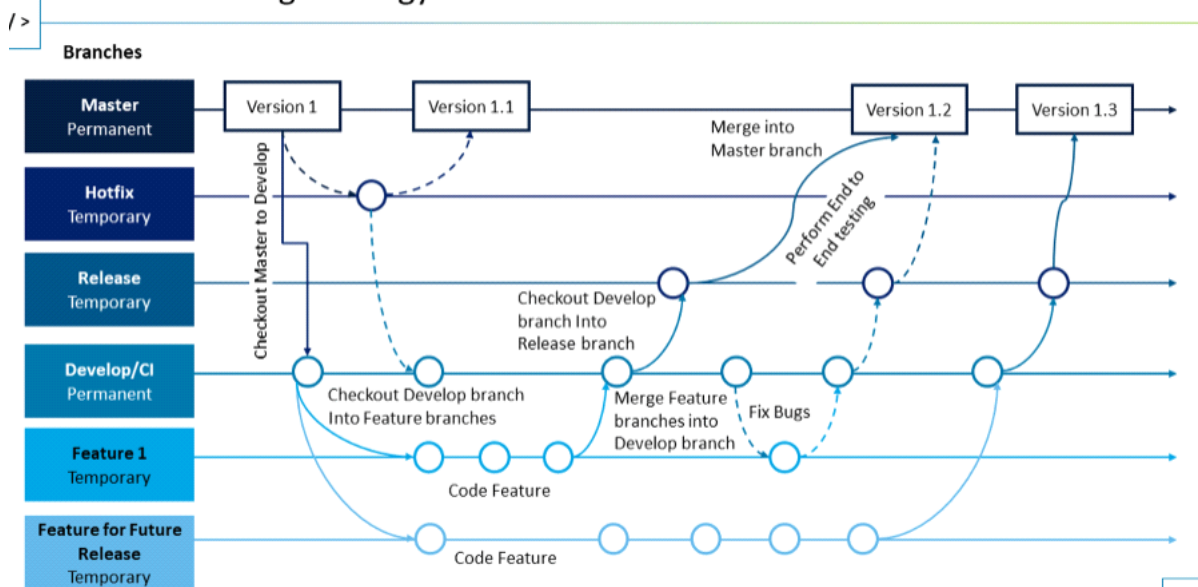
Git Workflow (Contd..)

1. Initializing a Repository: You start by creating a new Git repository or cloning an existing one.
2. Adding Changes: After making changes to your files, you stage them using `git add` to prepare them for committing.
3. Committing Changes: Once your changes are staged, you commit them to the repository using `git commit`. Each commit represents a snapshot of your code at a particular point in time.
4. Reviewing History: You can view the commit history using `git log` to see the changes made by you and others.
5. Branching and Merging: Create branches to work on new features or bug fixes. Use `git branch` to create branches and `git merge` to incorporate changes from one branch into another.
6. Working with Remotes: Git allows you to collaborate with others by pushing your changes to a remote repository (`git push`) and pulling changes from a remote repository (`git pull`).

Benefits of Git

- Collaboration: Git enables multiple developers to work on the same project simultaneously.
- Versioning: Git keeps track of all changes made to your code, allowing you to revert to previous versions if needed.
- Branching: Branches provide a way to isolate changes and work on them independently.
- Backup and Restore: Git repositories serve as a backup of your code, and you can restore previous versions easily.

GIT—Branching Strategy



Git command -help

Git help --all

Help commands

Git commit -help

Usage : git commit [] [--] ...

-q, --quiet suppress summary after successful commit

Git help -a-v, --verbo sell show diff in commit message template

Q exit and shift + g to jump at the end of the list.

JSTL TAGS AND EXPRESSION LANGUAGE(EL)

Wednesday, July 24, 2024 9:03 AM

To print something on the server we can use EL

JSTL CORE TAGS

To use jstl u need to use a jstl1.2 or any latest version jar file, but I don't know whether it's the case with maven

It should be inside webapp

If u keep session true, it will take a new session if there is no previous session then the username cannot be retrieved that's why we keep session false;

Generally when u forward a req dispatcher from doPost, it will go to doPost in the next servlet

When u do a sendRedirect it will go to the doGet

FORMAT OF THE EXPENSE PROJECT

MVC : MODEL VIEW CONTROLLER PATTERN

Model

User - Javabean

Expense - username - Javabean

UserController - validateuser / addUser

LogoutController - logout

ExpenseController - addexpense/getAllExpense

IUserService

validateUser(); - login

registerUser(); - register

IExpenseService

addExpense();

getAllExpenseByUserName();

IUserDAO

IExpenseDAO

HOW TO WORK WITH DATE IN JAVA

ExpenseBean -- private Date expenseDate

2022-04-24 -- String Date

String should be converted to date

Import DateFormat, Simple
DateFormat

import DateFormat, SimpleDateFormat, java.util.Date

```
Dateformat df=new SimpleDateFormat("yyyy-MM-ddHH:mm:ss");  
Date df=df.parse(request.getParameter("date"))  
setExpenseDate(df);
```

convert util.date to java.sql.Date

```
psmt.setDate(1,new java.sql.Date(bean.getExpenseDate().getTime()));
```

```
<%@page language="java" contentType="text/html; charset=UTF-8"  
pageEncoding="UTF-8"%>
```

```
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>  
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Expense page </title>
```

```
</head>
```

```
<body>
```

```
<table>
```

```
<tr>
```

```
<th>Username</th>
```

```
<th>Expense Date</th>
```

```
<th>Expense Type</th>
```

```
<th>Expense Description</th>
```

```
<th>Expense Amount </th>
```

```
</tr>
```

```
<tr>
```

```
<c:forEach items ="$(ListOfExpenses)" var = "s">
```

```
<td><%s.getu %>Hemanth</td>
```

```
<td>24-07-2024</td>
```

```
<td>Travel</td>
```

```
<td>Bus Fare</td>
```

```
<td>1500</td>
```

```
<c:forEach>
```

```
</table>
```

```
</body>
```

```
</html>
```

CODING FROM SCRATCH

Thursday, July 25, 2024 7:04 PM

IDEA : BATTER/BIKE/CAR

CAR TYPES -SUV,SEDAN,HACHBACK